

Lecture 1

Coding Foundations: Python, Git, & AI Agents

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Agosto 2026

Overview

1. Why Should a Geometer Code?
2. Python for Scientific Computing
 - The Ecosystem
 - Writing Good Scientific Code
3. Version Control with Git & GitHub
4. AI-Assisted Coding: Copilot & Agents
5. Course Workflow
6. Summary

Assumed background: None — this course expects no prior coding or AI experience; today explains the tools, and Tutorial 1 sets them up with you.

Guiding question: *What could computation do for your research that you currently do by hand?*

Why Should a Geometer Code?

Mathematics is (also) an experimental science

Computation lets us *generate examples*, *test conjectures*, and *search* spaces.

AI allows analysis at previously inaccessible scales, to find novel mathematical structure.

- **Conjecture generation:** ML-guided discovery of relations between knot invariants and in representation theory [Davies et al., *Nature* 2021].
- **Counterexample search:** Reinforcement learning constructions refuting conjectures in combinatorics [Wagner 2021].
- **Numerical geometry:** Neural-network approximations of Calabi–Yau metrics, where no closed form is known [Larfors et al., 2021].
- **Everyday research:** plotting, symbolic checks, dataset curation, literature search.

This course

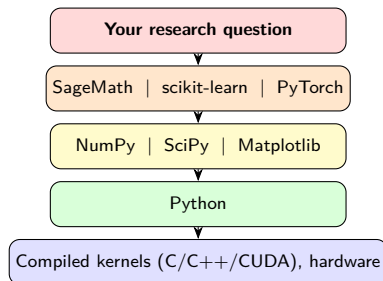
Each lecture introduces an AI technique; the paired tutorial applies it to *geometric datasets* (e.g. manifolds, varieties, graphs, simplicial complexes).

No coding experience assumed — today's tools are the foundations, LLMs will write your code.

Python: The Language of Scientific Computing

Why Python?

- Interpreted & readable:
code $\approx$ pseudo-code.
 - Enormous ecosystem:
a library for almost everything.
 - The interface language of AI:
PyTorch, JAX, TensorFlow.
 - Heavy lifting is done in compiled backends
— Python orchestrates.
-
- **NumPy**: arrays, linear algebra.
 - **Matplotlib**: visualisation.
 - **SciPy / scikit-learn**: numerics & classical ML.
 - **PyTorch**: differentiable programming, neural networks.
 - **SageMath**: computer algebra with geometry built in.



The scientific Python stack:
→ you write the top layers;
→ these use optimised lower layers.

What Good Scientific Code Looks Like

Principles (they mirror good mathematical writing)

- Name things meaningfully.
- Comment code clearly, describing intent (not mechanics).
- Keep lines short and clear.
- Make it work, then optimise.
- Isolate reusable steps into *functions*.
- Prefer *vectorised* array operations over explicit loops.

Computing pairwise distances of $N = 100$ points in $\mathbb{R}^3$: `X = np.random.rand(100,3)`

Example (Loops, slow)

```
D = np.zeros((100, 100))
for i in range(100):
    for j in range(100):
        diff = X[i] - X[j]
        D[i,j] = np.sqrt(diff @ diff)
```

Example (Vectorised, fast)

```
diff = X[:,None,:] - X[None,:,:]
D = np.linalg.norm(diff, axis=-1)
```

- Vectorised code is $65\times$ faster (often $10\text{--}1000\times$).
- Many package functions are already optimised.
- Thinking in arrays/tensors underlies all scientific computing.

Reproducibility: Computational Analogue of Proof

A computational claim is only as good as its reproducibility

An experiment someone else cannot rerun is like a proof no one can check.

Reproducibility rests on controlling three things: *code*, *environment*, *randomness*.

- **Environments:** Isolated installations with pinned library versions.
 - `conda create -n aigeo python=3.12` then `conda activate aigeo`
 - Record dependencies: `requirements.txt` / `environment.yml` lives in the repo.
- **Random seeds:** ML is stochastic (initialisation, shuffling, sampling). Fix seeds (`numpy`, `torch`) so results are repeatable; report average behaviour across several seeds.
- **Structure:** One repository per project — `data/`, `src/`, `notebooks/`, `results/`, `README`.

Good practice for the course

Every tutorial produces a repo that a colleague can clone and run with two actions:

- 1 Create the environment.
- 2 Execute the script.

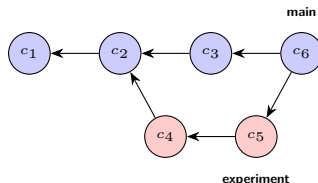
Git: The Framework for Distributed Code

Work History as a DAG

A repository's history is a DAG:

- Nodes are *commits*
(complete snapshots of codebase)
- Edges point to parents.
(last snapshot this was developed from)
- Branches* are separate lines of development;
merging joins branch histories.

- `git add + git commit`:
record a snapshot with a message.
- `git branch / git switch`:
parallel lines of development.
- `git merge`: combine branches;
conflicts are resolved explicitly.
- `git log / git diff`: interrogate
history — nothing is ever lost.



A feature branch (c_4, c_5) merged back into `main` at c_6 .

Cowork Analogy

Like writing a paper with a co-author:
You each draft sections separately
(branches), and reconcile them into the
master copy (merge), with the full edit
history preserved.

Note: overleaf has a git interface!

GitHub: Collaboration and Backup

GitHub = hosting for Git repositories + collaboration layer

Your local repository (project folder) synchronises with a *remote* repository:
`git push` uploads commits, `git pull` downloads collaborators' work.

- **Clone:** `git clone <url>` — copy any public repo (including this course's).
- **Pull requests:** propose changes; peers review the *diff* before merging — refereeing for code.
- **Issues:** structured to-do lists and bug reports tied to the code.
- **One repo per project:** each project's code is freely open-source, easy to reproduce results and develop.

Familiar analogy

Overleaf is essentially Git specialised to $\text{\LaTeX}$ (it even exposes a Git interface). GitHub is the same idea for arbitrary code, with far richer branching and review.

In Tutorial 1

You will create a GitHub account, clone the course repository, then test `{add, commit, push}` for your first changes.

AI Coding Assistants: What They Are

Large language models (LLMs) as coding partners

LLMs are large neural architectures trained on vast text/code corpora to predict the next token (based on the transformer mechanism in Lecture 6). Applied to code, they operate at three levels of autonomy:

- ① **Autocomplete** (e.g. Copilot inline): suggests the next lines as you type.
- ② **Chat**: you describe a task in natural language; it drafts code, explains errors, translates mathematics into NumPy/PyTorch.
- ③ **Agents** (e.g. Copilot agent mode in VSCode): given a goal, they iterate — write code, run it, read the errors, fix, repeat — across multiple files.
- Consequence: *syntax is no longer the barrier to entry*.
Your mathematical precision in specifying the problem is now the valuable skill.
- This course uses **VSCode + GitHub Copilot** (free for academics) as the standard setup, but we will setup and demonstrate ChatGPT **Codex** & Anthropic's **Claude** too; demonstrated in Tutorial 1.

Using Agents Properly

Golden rule

An agent is a brilliant, tireless, occasionally overconfident collaborator; which will enormously accelerate your workflow. This course will develop responsible use of them, verification is essential – your mathematician's scepticism is your comparative advantage.

Do

- Give precise context: definitions, conventions, expected input/output shapes.
- Work in small, verifiable steps; commit after each working step.
- Ask for tests and sanity checks (e.g. “verify on S^2 , where the answer is known”).
- Ask it to *explain* its code — and read the explanation.

Avoid

- Accepting long generated code you have not read.
- Vague prompts (“make it work”) — garbage in, garbage out.
- Letting an agent loose without version control: Git is your undo button.
- Assuming numerical output is correct because it ran without errors.

Effective prompt pattern

Role + context + task + constraints + verification: “You are an expert mathematician assisting with differential geometry computations. Write a NumPy function sampling N points uniformly on $S^2 \subset \mathbb{R}^3$. Input: N , seed. Output: array($N,3$). Include a test verifying point norms = 1.”

How This Course Works

Structure: 13 lectures + 13 tutorials over 8 weeks

Lectures give the mathematical structure of each AI technique; tutorials implement it on geometric data. Assessment: an individual project applying AI to a geometric problem.

- **Common data theme:** sampled manifolds (S^n , tori, hypersurfaces), algebraic varieties, graphs/triangulations, discretised operators — objects you already understand, so the ML behaviour is interpretable.
- **Course repository:** all tutorial notebooks, datasets, and these slides; you fork it and build your own solutions on branches.
- **Roadmap:**
 - Week 1:** Foundations (L1–2)
 - Week 2-3:** Supervised learning (L3–5)
 - Week 3:** Transformers (L6)
 - Week 4:** Unsupervised learning (L7–8)
 - Week 5:** Reinforcement learning (L9)
 - Week 5-6:** Geometry-aware ML (L10–11)
 - Week 6-7:** Neural PDE solvers (L12–13)
 - Week 7-8:** Projects & Presentations
- **Project seeds** appear throughout — note what excites you; projects can grow into research papers following the course!

Summary

Key points

- AI is now a genuine tool for geometry: conjecture generation, counterexample search, numerical approximation of geometric quantities.
- Python + NumPy/PyTorch is the standard stack; data is *arrays and tensors*.
- Reproducibility = code (Git) + environment (conda) + controlled randomness (seeds).
- Git is a DAG of snapshots; GitHub adds collaboration, review, and open-source.
- AI agents remove the syntax barrier — use them in small verified steps, with Git as the safety net.

Next

Tutorial 1: Install Python/conda, VSCode + Copilot, create your GitHub account and clone this repo. First code computations with point clouds on S^1, S^2 .

Lecture 2: The mathematical foundations of machine learning.